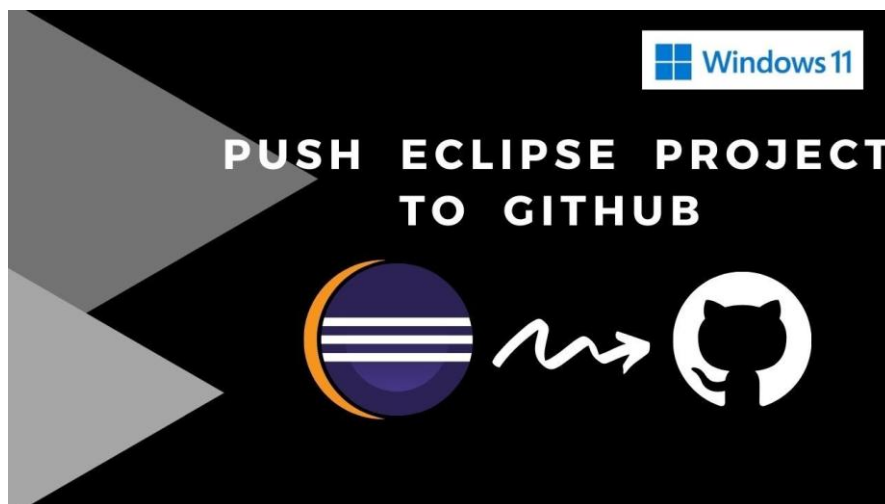


Industrial Tool

Git & Github

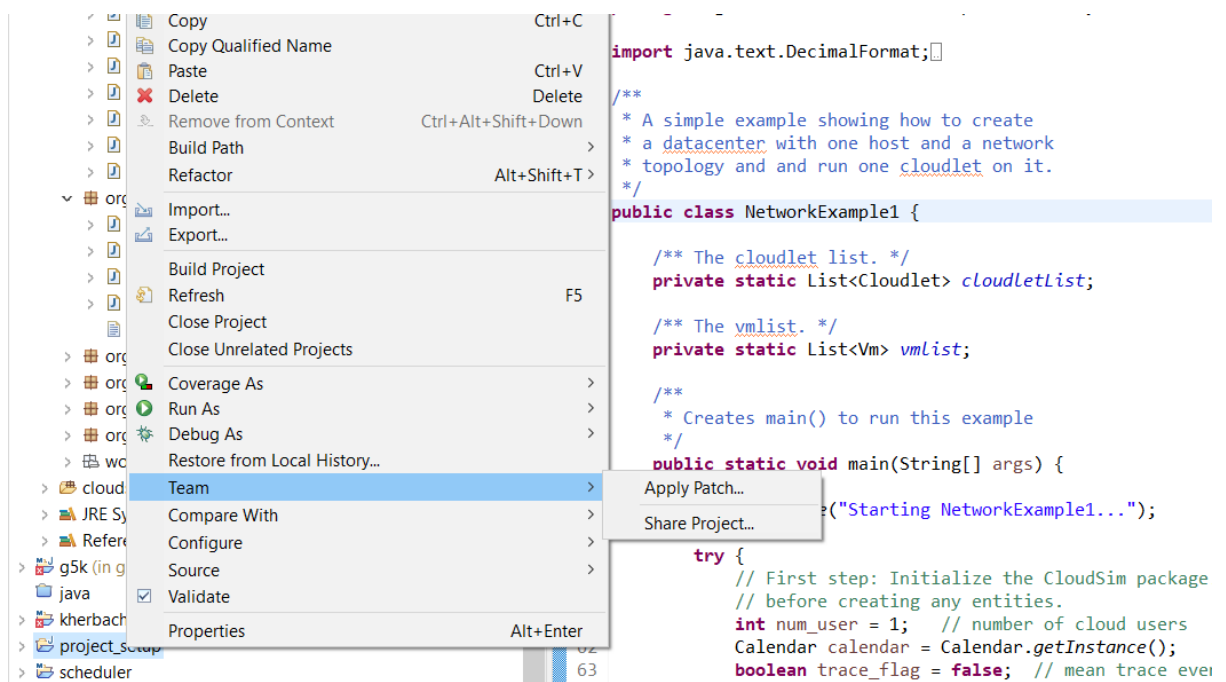
How to Export Eclipse projects to GitHub?

- Eclipse is an Integrated Development Environment (IDE) for programming.
- It has a base workspace and an extensible plug-in system for customization.
- Eclipse is written mainly in Java.
- Its primary use is Java application development, but it supports other languages via plug-ins.

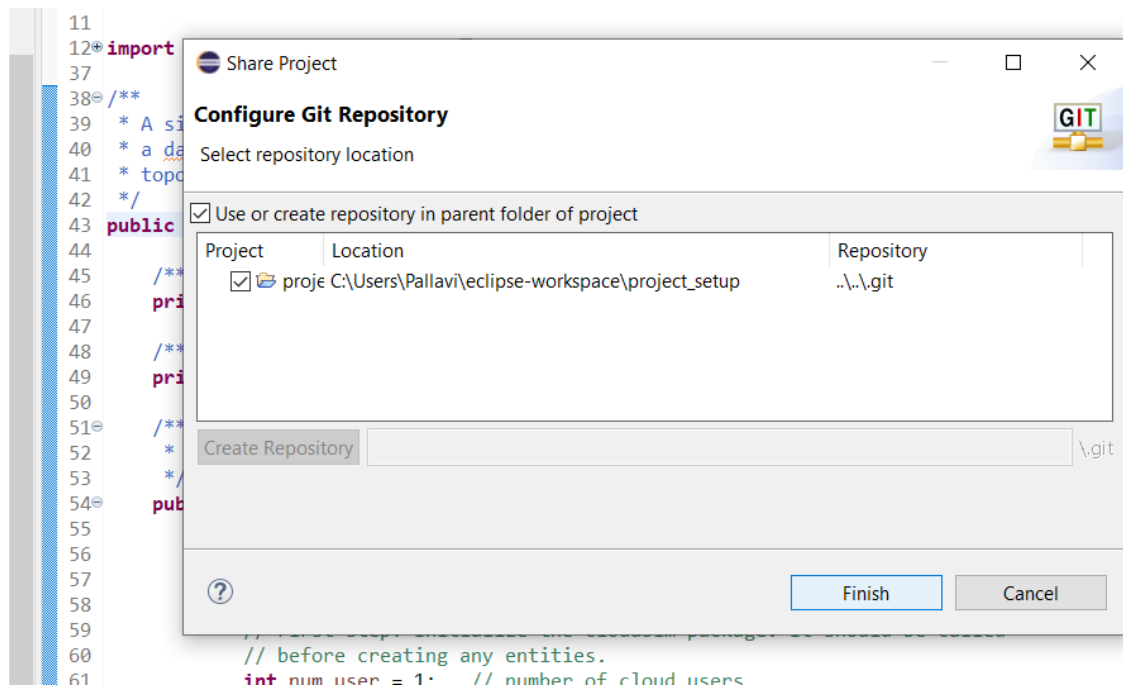


Step by Step Implementation :-

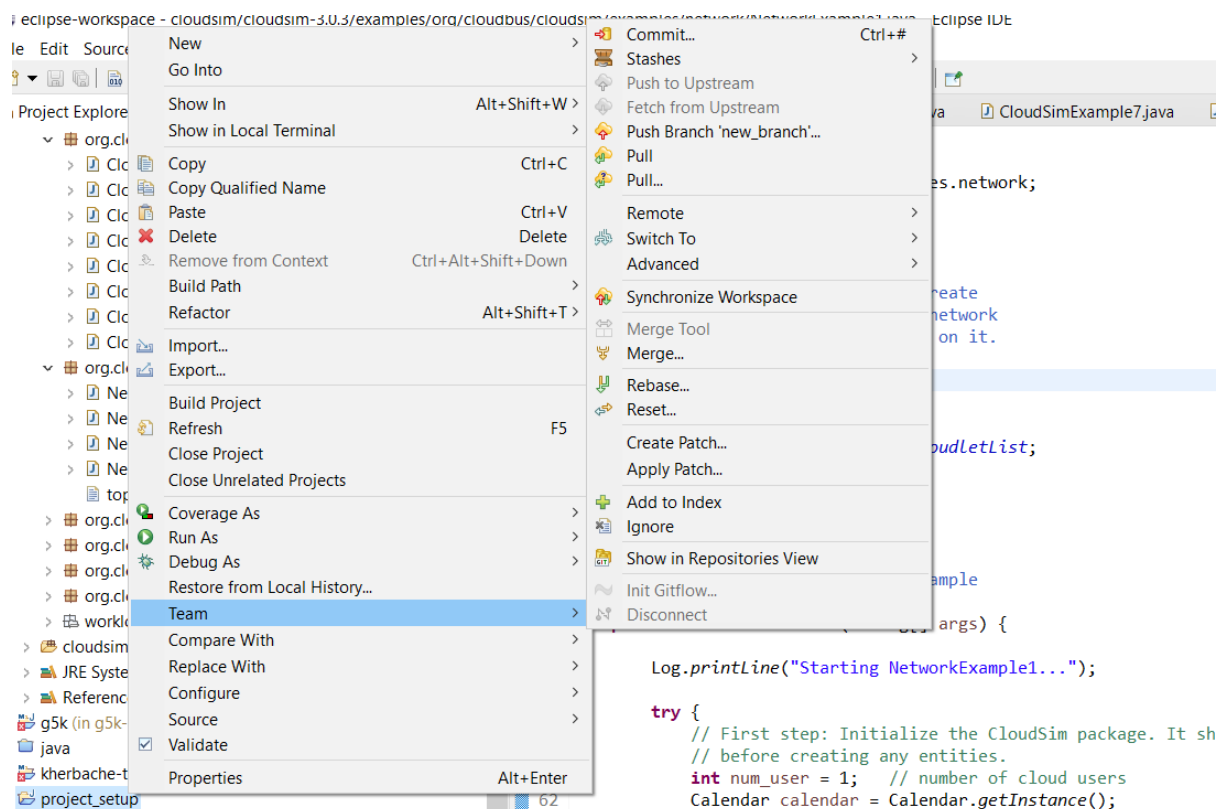
Step 1: Open Eclipse IDE and right-click on the project you want to push and go to **Team - share project**.



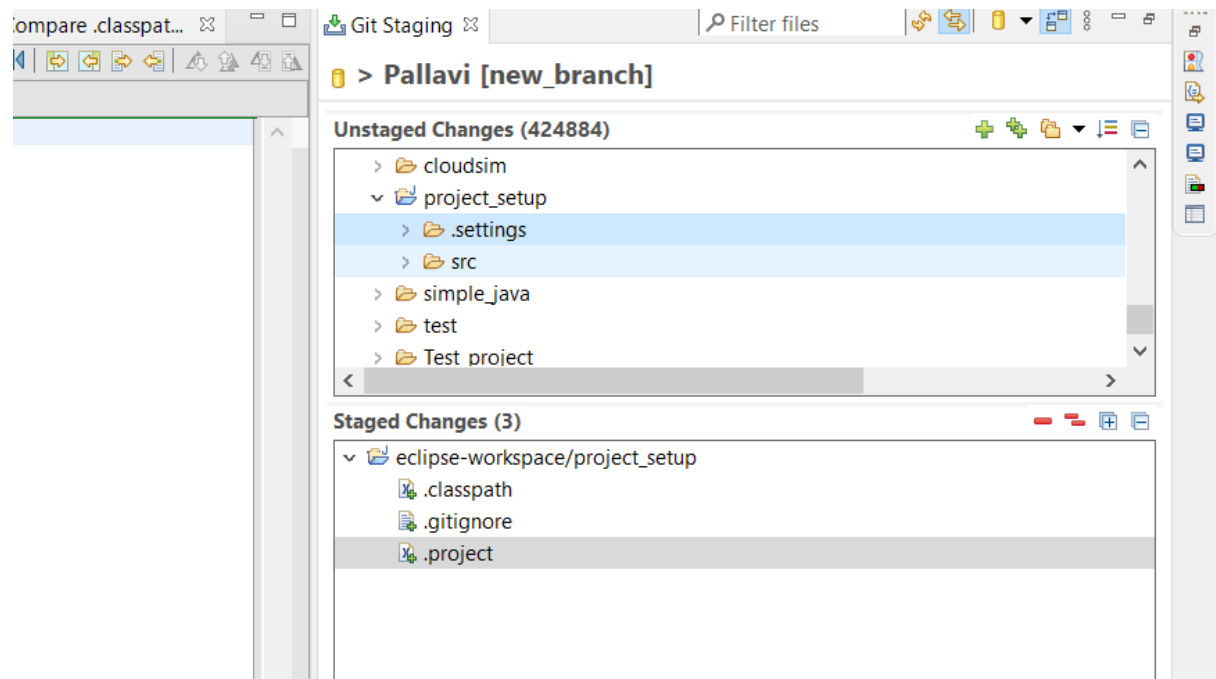
Step 2: It will add the project to the given repository as shown below:



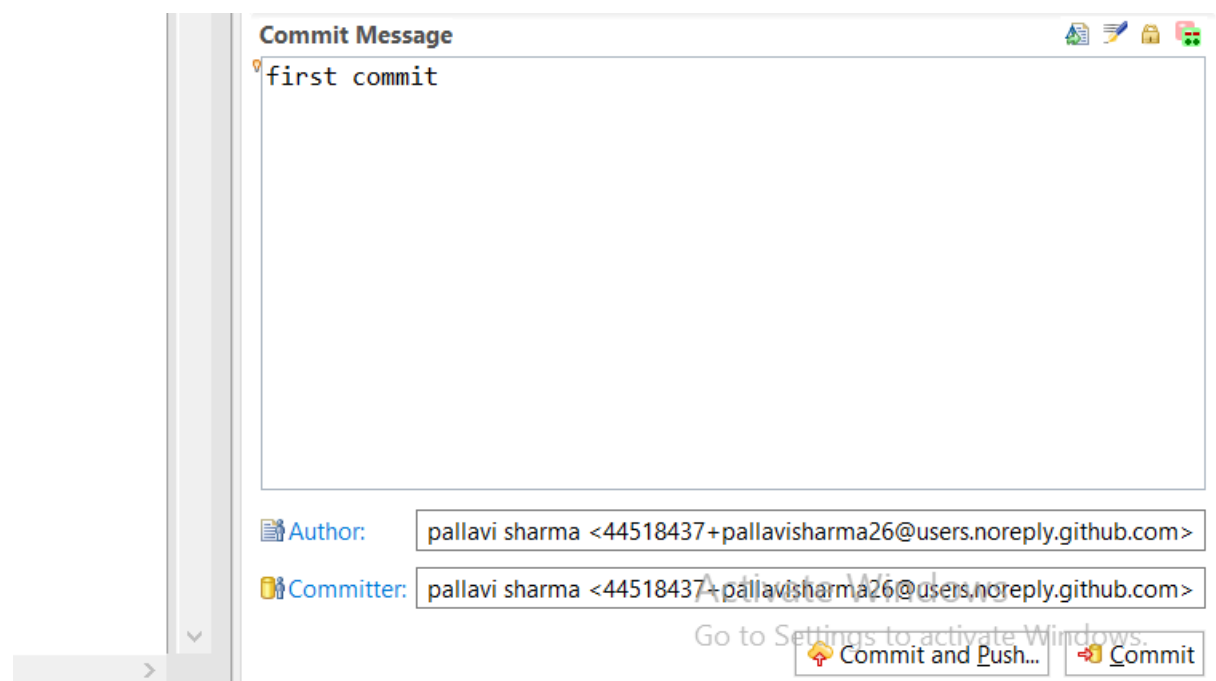
Step 3: Again right-click on the project and go to **Team->commit**.



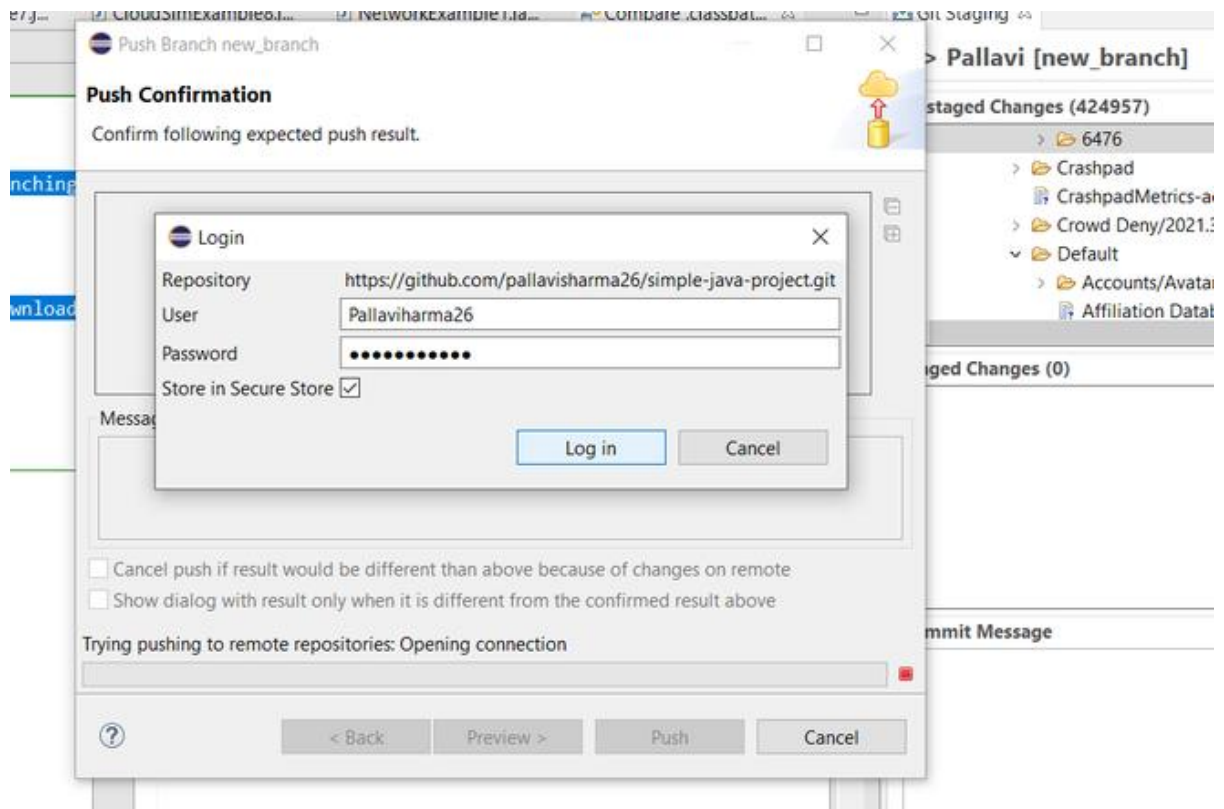
Step 4: Drag and Drop the files you want to commit from **Unchanged Changes** to **Staged Changes**.



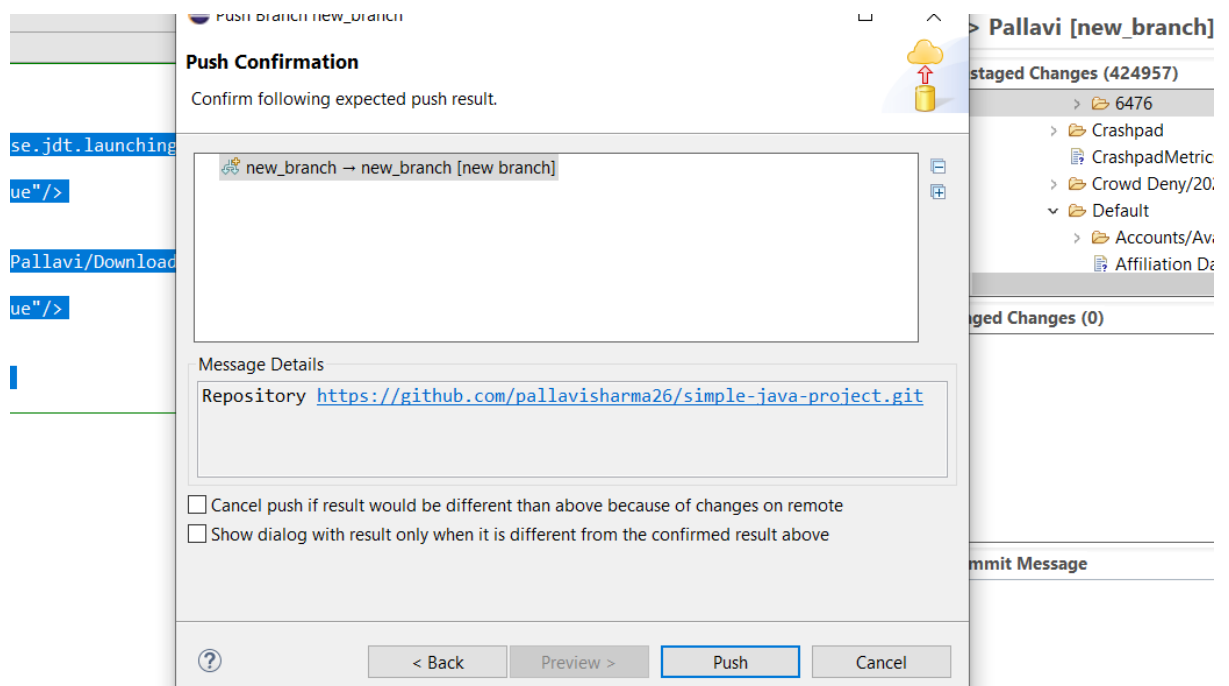
Step 5: Write the commit message in "**Commit Message**" and click "**Commit and Push**".



Step 6: Fill in your UserID and password of GitHub and click "**Log in**".



Step 7: Push to the new-branch of GitHub Repository and click "**Push**".



The new branch is created on GitHub and is required to be merged by creating a pull request.

26 / simple-java-project

simple-java-project

requests

Actions

Projects

Wiki

Security

Insights

Settings

new_branch had recent pushes less than a minute ago

Compare & pull request

master

2 branches

0 tags

Go to file

Add file

Code

this branch is even with buddy-works:master.

Pull request

Compare

sizix a

2c8575a on Aug 19, 2019

5 commits

.github	a	2 years ago
src	Fixing unit test failure for demo	5 years ago
.gitignore	simple java project	5 years ago
Procfile	simple java project	5 years ago
README.md	Update README.md	5 years ago

Git Logs :-

- Git log shows the complete commit history.
- It helps track changes over time.
- It lets you review project progress.
- It shows who made changes and when.

Git log displays a list of all commits made in a repository in reverse chronological order (latest commit first).

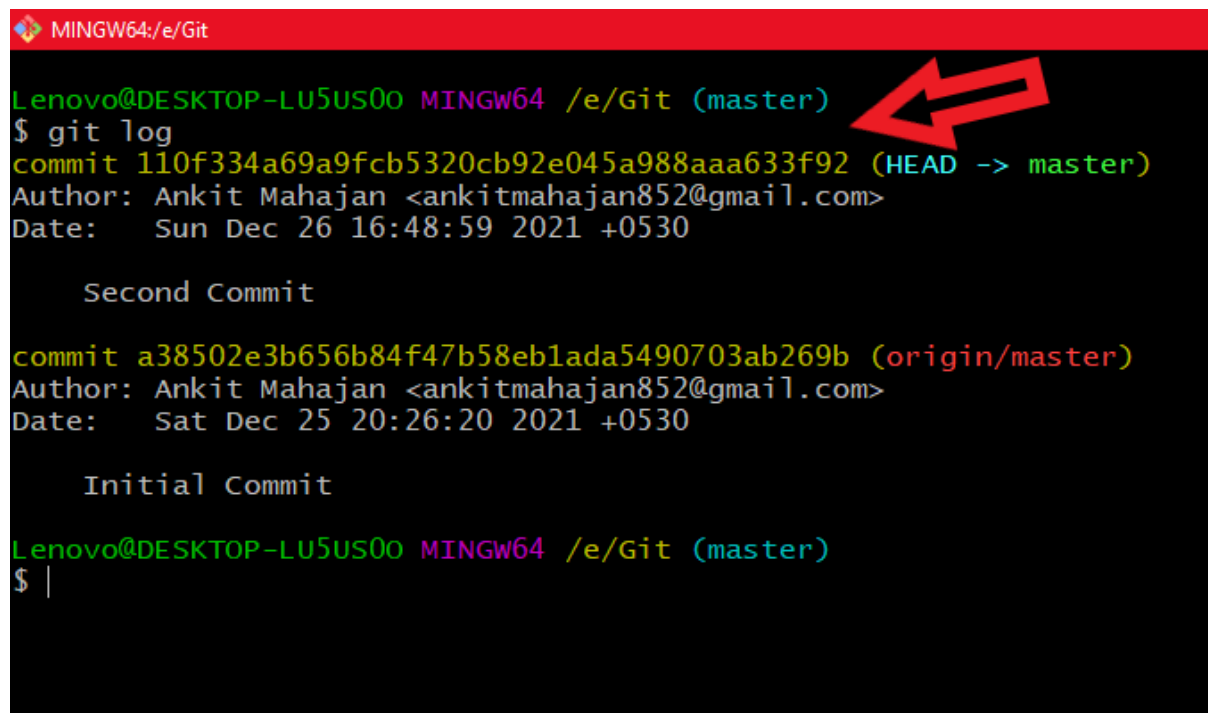
It shows detailed information about each commit, including

- Commit hash (ID) – a unique identifier for each commit.
- Author name and email – who made the commit.
- Date and time – when the commit was made.
- Commit message – a short description of the change

Basic Syntax :-

git log

Example output:



```
MINGW64:/e/Git
Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ git log
commit 110f334a69a9fcb5320cb92e045a988aaa633f92 (HEAD -> master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sun Dec 26 16:48:59 2021 +0530

    Second Commit

commit a38502e3b656b84f47b58eb1ada5490703ab269b (origin/master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sat Dec 25 20:26:20 2021 +0530

    Initial Commit

Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ |
```

git log

Each entry represents one commit.

Useful Git Log Options :-

Git Log has many options to customize how you view the commit history.

Here are some of the most commonly used ones:

1. View a Compact One-Line Log

Shows each commit in a single line (commit hash and message only).

git log --oneline

2. Show Graph of Branches :-

Displays a visual branch structure along with commits.

git log --oneline --graph

Helps to understand how different branches have diverged and merged.

3. Show Commits by a Specific Author :-

View only commits made by a particular developer.

git log --author="Alice"

Useful for reviewing contributions by a specific team member.

4. Filter Commits by Date :-

View commits within a certain date range.

git log --after="2025-09-01" --before="2025-10-01"

Helps track progress during specific development periods.

5. Search Commits by Message Keyword :-

Find commits with specific keywords in their message.

git log --grep="bug fix"

Useful for locating when a specific feature or fix was Introduced.

6. View Files Changed in Each Commit :-

Show the list of modified files in each commit.

git log --stat

Displays file-level changes and total insertions/deletions.

7. See the Actual Code Changes :-

To view the full diff (code changes) in each commit:

git log -p

This is very detailed — great for code review and debugging.

Log Search : Finding Specific Commits :-

Git provides several ways to search through commit history, making it easier to locate specific changes, messages, authors, or dates.

1. Search by Commit Message

- Use `--grep` to find commits by keywords.
- Example: `git log --grep="fix bug"`.
- Useful for tracking fixes or features.

2. Search by Author

- Filters commits by a specific developer.
- Example: `git log --author="Alice"`.
- Helpful for reviewing individual contributions.

3. Search by Date

- Filters commits within a date range.
- Example: `git log --after="2025-09-01" --before="2025-10-01"`.
- Useful for sprint or phase tracking.

4. Search by File Changes

- Shows commits that modified a specific file.
- Example: `git log -- <file-path>`.
- Helps track file-level history.

5. Search by Commit Hash

- Search using full or partial commit hash.
- Example: `git log a1b2c3`.
- Displays the matching commit details.

6. Combining Search Options

- Multiple filters can be used together.
- Example: `author + message + date`.
- Provides precise commit results.

7. git shortlog

Basically it displays the log output format in the following syntax:

Committer1(number of commits)

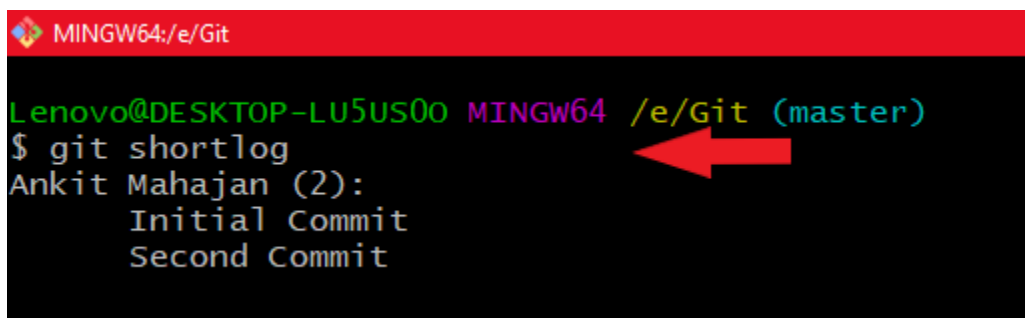
- Commit Message 1
- Commit Message 2
- Commit Message n

where n represents the last commit message number committed by the author

Committer2(number of commits)

- Commit Message 1
- Commit Message 2
- Commit Message n

where n represents the last commit message number committed by the author

A screenshot of a Windows command prompt window with a red title bar that says "MINGW64:/e/Git". The prompt shows the user "Lenovo@DESKTOP-LU5US00" in a "MINGW64" environment at the directory "/e/Git" on the "master" branch. The command "\$ git shortlog" has been executed, resulting in the output: "Ankit Mahajan (2):", "Initial Commit", and "Second Commit". A red arrow points from the right towards the output text.

Using git shortlog

Here, only my commits appear because I am the sole contributor.

In larger projects, **git shortlog** shows multiple contributors along with their commit counts and messages.

Parameter	Details
-s or --summary	It displays the number of commits and the committer's name
-e or --email	It displays the committer's name, number of commits, commit messages along with the mail id
-n or --numbered	It sorts the output by the number of commits instead of alphabetically by committer's name

8. git grep

- git grep searches for a specific string in files or commits.
- It helps quickly locate where a string exists in the repository.

```
MINGW64:/e/Git
Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ git grep hello
testing-git:hello
Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ git grep "hello cloud" a38502e3b656b84f47b58eb1ada5490703ab269b
a38502e3b656b84f47b58eb1ada5490703ab269b:testing-git:hello cloud engineers
```

Using git grep

For a range of lines within a file: The command would be the same git log just some new parameters we have to pass in it for checking for a range of lines within a file. The command is:

git log -L starting_line_number,ending_line_number:file_name here -L denotes the line range for a file.

```
MINGW64:/e/Git
Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ git log -L 1,3:testing-git
commit 110f334a69a9fcb5320cb92e045a988aaa633f92 (HEAD -> master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sun Dec 26 16:48:59 2021 +0530

    Second Commit

diff --git a/testing-git b/testing-git
--- a/testing-git
+++ b/testing-git
@@ -1,1 +1,3 @@
-hello cloud engineers
+hello
+cloud
+engineers

commit a38502e3b656b84f47b58eb1ada5490703ab269b (origin/master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sat Dec 25 20:26:20 2021 +0530

    Initial Commit

diff --git a/testing-git b/testing-git
--- /dev/null
+++ b/testing-git
@@ -0,0 +1,1 @@
+hello cloud engineers
```

Using git log -L starting_line_number,ending_line_number:file_name

9. Filter logs:

In this concept, we will see various parameters used in the command to filter logs.

- For some days: **git log --after 'mention the days ago'**
- for e.g: **git log --after '4 days ago'**
- For specific dates: **git log --after year_number-month_number-date_number**
- For Author name: **git log --author="Author_name"**

We can also use the **since** clause instead of after in this command.

```
MINGW64:/e/Git
Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ git log --after='7 days ago'
commit 110f334a69a9fcb5320cb92e045a988aaa633f92 (HEAD -> master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sun Dec 26 16:48:59 2021 +0530

Second Commit

Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ git log --after='8 days ago'
commit 110f334a69a9fcb5320cb92e045a988aaa633f92 (HEAD -> master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sun Dec 26 16:48:59 2021 +0530

Second Commit

commit a38502e3b656b84f47b58eb1ada5490703ab269b (origin/master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sat Dec 25 20:26:20 2021 +0530

Initial Commit
```

Using git log --after 'number_of_days days ago'

```
MINGW64:/e/Git
Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ git log --after 2021-12-25
commit 110f334a69a9fcb5320cb92e045a988aaa633f92 (HEAD -> master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sun Dec 26 16:48:59 2021 +0530

Second Commit

commit a38502e3b656b84f47b58eb1ada5490703ab269b (origin/master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sat Dec 25 20:26:20 2021 +0530

Initial Commit

Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ git log --after 2021-12-26
commit 110f334a69a9fcb5320cb92e045a988aaa633f92 (HEAD -> master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sun Dec 26 16:48:59 2021 +0530

Second Commit
```

Using git log --after year_number-month_number-Date_number

```
MINGW64:/e/Git

Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ git log --author="Ankit Mahajan"
commit 110f334a69a9fcb5320cb92e045a988aaa633f92 (HEAD -> master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sun Dec 26 16:48:59 2021 +0530

    Second Commit

commit a38502e3b656b84f47b58eb1ada5490703ab269b (origin/master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sat Dec 25 20:26:20 2021 +0530

    Initial Commit
```

Using git log --author="author-name"

10. Changes inline

If our use-case is to see the logs with changes inline we use a parameter -p or --patch to see the logs with changes inline.

```
MINGW64:/e/Git

Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ git log --patch
commit 110f334a69a9fcb5320cb92e045a988aaa633f92 (HEAD -> master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sun Dec 26 16:48:59 2021 +0530

    Second Commit

diff --git a/testing-git b/testing-git
index 2067cc5..fc6378b 100644
--- a/testing-git
+++ b/testing-git
@@ -1,2 +1,4 @@
-hello cloud engineers
+hello
+cloud
+engineers

commit a38502e3b656b84f47b58eb1ada5490703ab269b (origin/master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sat Dec 25 20:26:20 2021 +0530

    Initial Commit

diff --git a/testing-git b/testing-git
new file mode 100644
index 0000000..2067cc5
--- /dev/null
+++ b/testing-git
@@ -0,0 +1,2 @@
+hello cloud engineers
```

Using git log --patch

11. Committed files

If you want to see the files in logs that are being committed by commits. we use the command

git log --stat

A terminal window titled 'MINGW64:/e/Git' showing the output of 'git log --stat'. The output lists three commits. A red arrow points to the first commit's hash '110f334a69a9fcb5320cb92e045a988aaa633f92'. A red box highlights the 'Second Commit' section, which shows a diff for 'testing-git' with 4 changes (3 insertions, 1 deletion). The box also highlights the 'Initial Commit' section, which shows a diff for 'testing-git' with 2 insertions. The terminal text is as follows:

```
Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ git log --stat
commit 110f334a69a9fcb5320cb92e045a988aaa633f92 (HEAD -> master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sun Dec 26 16:48:59 2021 +0530

    Second Commit

testing-git | 4 +++-
1 file changed, 3 insertions(+), 1 deletion(-)

commit a38502e3b656b84f47b58eb1ada5490703ab269b (origin/master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sat Dec 25 20:26:20 2021 +0530

    Initial Commit

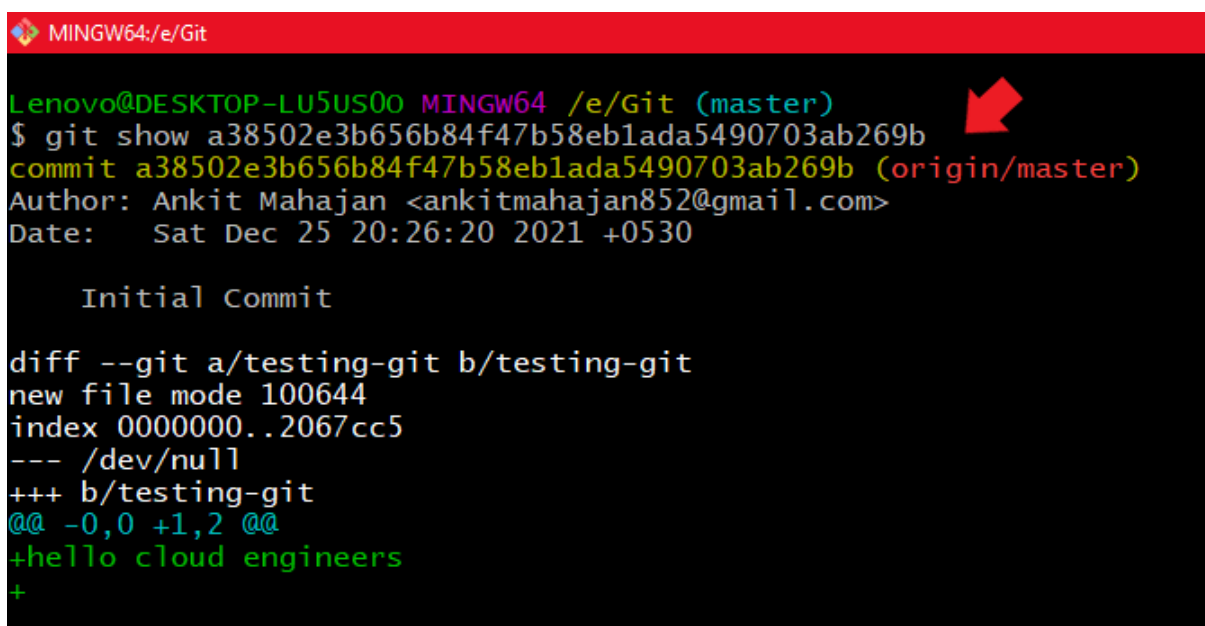
testing-git | 2 ++
1 file changed, 2 insertions(+)
```

Using git log --stat

Contents of a Commit:

Now if you want to view the contents of a commit you can use a simple command

git show commit-id

A terminal window titled 'MINGW64:/e/Git' showing the output of 'git show' for a specific commit. A red arrow points to the commit hash 'a38502e3b656b84f47b58eb1ada5490703ab269b'. The output shows the commit details and a diff for 'testing-git' showing two insertions: 'hello' and 'cloud engineers'. The terminal text is as follows:

```
Lenovo@DESKTOP-LU5US00 MINGW64 /e/Git (master)
$ git show a38502e3b656b84f47b58eb1ada5490703ab269b
commit a38502e3b656b84f47b58eb1ada5490703ab269b (origin/master)
Author: Ankit Mahajan <ankitmahajan852@gmail.com>
Date: Sat Dec 25 20:26:20 2021 +0530

    Initial Commit

diff --git a/testing-git b/testing-git
new file mode 100644
index 0000000..2067cc5
--- /dev/null
+++ b/testing-git
@@ -0,0 +1,2 @@
+hello cloud engineers
+
```

Using git show commit-id

Committer name and time since commit

Now if you want to see the committer name and time since he committed in oneline. We use the following command

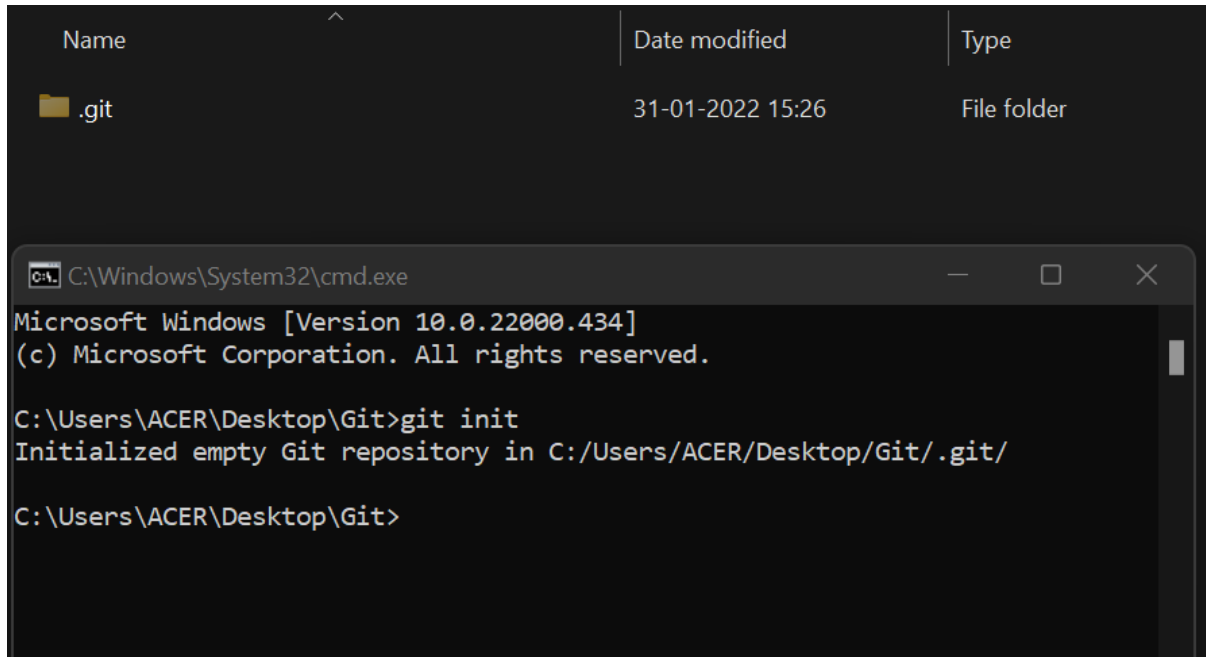
```
git log --oneline --decorate --source --pretty=format:"%Cblue %h %Cgreen %ar %Cblue  
%an %C(yellow) %d %Creset %s" --all --graph
```

```
* " 110f334 7 days ago Ankit Mahajan (HEAD -> master) Second Commit"  
* " a38502e 8 days ago Ankit Mahajan (origin/master) Initial Commit"
```

The output of the above command

Git - git-show Command Line Utility :-

- We use Git daily, but some commands are still less known.
- One such command is git show.
- Before using it, we initialize a local directory with git init to create a Git repository.



- In Git, the .git folder stores repository data.
 - The .git/objects directory holds different objects like blobs, trees, commits, and tags.
1. Blob Object - stores the contents of the file
 2. Tree Object - contains a list of all files in our repository with a pointer to the blob object assigned to them
 3. Commit Object - points a pointer to its tree object
 4. Tag Object - shows the tag message and other objects included in the tag (object name, object type, tag name)

To view expanded details on these objects we use the command **git show**. Let us understand more about the command with the help of an example. We have created a file named **demo.txt**, wrote 2 lines in it, and committed the changes.

```
C:\Users\ACER\Desktop\Git>git log
commit 9e200dee910f9e668ab80a3c9e4fedfc5654e29f (HEAD -> master)
Author: pikachua7 <atharvapaliwal7@gmail.com>
Date:   Fri Jan 28 11:26:00 2022 +0530

    Added Hello World

commit c5c7cd942a4136f12878efe1c719911b5f2fa258
Author: pikachua7 <atharvapaliwal7@gmail.com>
Date:   Fri Jan 28 11:25:26 2022 +0530

    Added Hello Geeks
```

Let's now use **git show** command and see what all information we get using the command.

git show <commit id>

```
C:\Users\ACER\Desktop\Git>git show 9e200dee910f9e668ab80a3c9e4fedfc5654e29f
commit 9e200dee910f9e668ab80a3c9e4fedfc5654e29f (HEAD -> master)
Author: pikachua7 <atharvapaliwal7@gmail.com>
Date:   Fri Jan 28 11:26:00 2022 +0530

    Added Hello World

diff --git a/demo.txt b/demo.txt
index 663aba4..d604583 100644
--- a/demo.txt
+++ b/demo.txt
@@ -1,2 @@
-Hello Geeks
\ No newline at end of file
+Hello Geeks
+Hello World
\ No newline at end of file

C:\Users\ACER\Desktop\Git>
```

We can see that there is a lot of information that we got when we use **git show** command. From the above image, we can infer that **git show** command shows us 2 things

- **Part 1:** The commit message and the pointer to which the HEAD is pointing.
- **Part 2:** Second thing that can see is the different versions of the file .

Let's discuss the 2 parts to some depth as follows:

PART 1

```
commit 9e200dee910f9e668ab80a3c9e4fedfc5654e29f (HEAD -> master)
Author: pikachua7 <atharvapaliwal7@gmail.com>
Date:   Fri Jan 28 11:26:00 2022 +0530

    Added Hello World

C:\Users\ACER\Desktop\Git>git log
commit 9e200dee910f9e668ab80a3c9e4fedfc5654e29f (HEAD -> master)
Author: pikachua7 <atharvapaliwal7@gmail.com>
Date:   Fri Jan 28 11:26:00 2022 +0530

    Added Hello World
```

The first part gives us the same result as when we type the **git log** command which tells us about the commit history of the commit id.

HEAD -> master

Tells us about the pointer where the HEAD is currently pointing.

Since this commit had a pointer to the HEAD, let's see another example of **git show** where the commit doesn't have a pointer to the HEAD.

```
C:\Users\ACER\Desktop\Git>git show c5c7cd942a4136f12878efe1c719911b5f2fa258
commit c5c7cd942a4136f12878efe1c719911b5f2fa258
Author: pikachua7 <atharvapaliwal7@gmail.com>
Date:   Fri Jan 28 11:25:26 2022 +0530

    Added Hello Geeks

diff --git a/demo.txt b/demo.txt
new file mode 100644
index 00000000..663aba4
--- /dev/null
+++ b/demo.txt
@@ -0,0 +1 @@
+Hello Geeks
\ No newline at end of file
```

We didn't get **HEAD->master** here as this commit is not pointing to the HEAD.

PART-2

```
diff --git a/demo.txt b/demo.txt
index 663aba4..d604583 100644
--- a/demo.txt
+++ b/demo.txt
@@ -1,2 @@
-Hello Geeks
\ No newline at end of file
+Hello Geeks
+Hello World
\ No newline at end of file
```